

Test Documentation – “Give the Duck a Qwacker”

Project: Give the Duck a Qwacker

Tester: Darlene Couch

Test Dates: 11–14 May 2026

Notes:

- This document covers unit, system, and user testing.
- All tests include exact actions, expected outcomes, and actual results.
- Remediation actions are recorded for failed tests and retested.

Test Strategy

- **Unit Testing:** Test individual class methods (Duck, Cracker, Questions) to ensure correct outcome.
- **System Testing:** Test the fully integrated game, including UI, scoring, timer, and animations.
- **User Testing:** Test from the player’s perspective, checking usability, question locking, score updates, win/lose conditions, and feedback.
- **Exceptional / Extreme / Normal:** Each test is shown as below.

System / User Testing

Test No	Action	Expected Outcome	Actual Outcome	Result (Pass/Fail)	Test Type
1	Launch game from welcome screen	Game window opens, background displayed, score = 0, timer = 50	As expected	Pass	Normal
2	First question appears	Question text visible with three answer buttons	As expected	Pass	Normal
3	Click correct answer	Score increases by 1, “Correct! Feed your Duck” shows, correct sound effect plays, duck moves toward cracker, cracker displayed	As expected	Pass	Normal
4	Click wrong answer	Score unchanged, “Wrong! Try again” shows, wrong sound effect plays, score does not change	As expected	Pass	Normal
5	Click multiple answers quickly	Only first answer processed; question locked until next question	Score increased with multiple clicks	Fail	Extreme
6	Reach 10 correct answers	Score = 10, “You Win!” message displayed, win sound effect plays, duck jumps and then floats, game over triggered	As expected	Pass	Normal

Test No	Action	Expected Outcome	Actual Outcome	Result (Pass/Fail)	Test Type
7	Let timer reach 0	"Times Up!" message displayed, lose sound effects plays, game stops, game over screen opens	Game continued running (no game over trigger)	Fail	Extreme
8	Observe duck/cracker animation	Duck moves toward cracker, crunch sound effects plays, cracker disappears after collision	As expected	Pass	Normal
9	Invalid input (click outside answer buttons)	Input ignored, no crash	As expected	Pass	Normal
10	Close game window mid-play	Window closes and welcome screen displays	As expected	Pass	Normal
11	Background image missing	Game starts, default background file loaded on the canvas	No background loaded	Fail	Exceptional
12	If Player name not entered	Name defaults to "Player", saves to CSV, displays on leaderboard	No names displaying	Fail	Exceptional
13	Win screen triggered multiple times	Only one win screen opens	Multiple screens opening	Fail	Extreme
14	Duck stops on ground after movement	Duck Y coordinate set to self.ground_y after moving toward cracker or pond jump	Duck stops in mid-air (floating)	Fail	Normal

Test Remediation

Retest Date	Test No	Action	Issue	Remediation Action	Result
14th May 2026	5	Click multiple answers quickly	Previously multiple clicks updated score incorrectly	Added self.question_locked = True until next question loads	Pass
14th May 2026	7	Timer reaches 0	Timer reached 0 but game did not stop immediately	Fixed condition in update_timer() to set game_over = True and hide questions	Pass

Retest Date	Test No	Action	Issue	Remediation Action	Result
14th May 2026	11	Background image missing	Background image failed to display	Verified PNG file exists, self.bg_img = Image.open("bg.png") and loaded on canvas	Pass
14th May 2026	12	Player name not saving	Name did not save to CSV or display	Ensure save_score() called on Submit, update self.name_text	Pass
14th May 2026	13	Win screen opened multiple times	Multiple win screens appeared	Added if self.game_over: return at start of win_game()	Pass
14th May 2026	14	Duck floating above ground	Duck did not land at ground level	Explicitly set new_y = self.ground_y when target reached correct co-ordinates	Pass

Summary / Observations

- All core functionality passed after minor fixes.¹
- System tests confirm integration of classes and game flow works correctly (score updates, question display, animations).
- User testing confirms game is playable, feedback is correct, and win/lose conditions trigger appropriately.²
- Minor issues with question locking and timer end were remediated and retested successfully.

¹ See appendix 1 for code examples and fixes.

² See appendix 2 for debug example and trace table

Appendix 1

Multiple Answer Clicks Increased Score More Than Once

Problem

Players could spam-click answer buttons quickly and gain multiple points from one question causing the duck to do the final pond jump before reaching the end trigger for game over.

Cause

The question buttons were not locked immediately after a click, allowing multiple triggers increasing the score prematurely.

Buggy Code

```
# Function for correct answer logic
def correct_answer(self): 1 usage
    # Prevents multiple clicks being processed
    if self.question_locked or self.game_over:
        return

    # Shows the correct answer feedback on the canvas
    self.result_label.config(text="Correct! Feed your Duck", font=("Arial", 14), bg="light blue")
    self.result_label.place(x=350, y=320, anchor="center")

    # Increases the score by 1
    self.score += 1

    # Updates the score display on the canvas
    self.canvas.itemconfig(self.score_text, text=f"Score = {self.score}")
    print(f"score {self.score}")

    # Checks the win condition for 10 correct answers and then calls the win_game function
    if self.score == 10:
        self.win_game()
        return

    # Plays the correct answer sound effect
    play_correct()
```

Fix

Add a lock (self.question_locked) to prevent multiple answers from being processed.

Fixed Code

```
# Function for correct answer logic
def correct_answer(self): 1 usage
    # Prevents multiple clicks being processed
    if self.question_locked or self.game_over:
        return

    # Shows the correct answer feedback on the canvas
    self.result_label.config(text="Correct! Feed your Duck", font=("Arial", 14), bg="light blue")
    self.result_label.place(x=350, y=320, anchor="center")
    self.question_locked = True # locks the question

    # Increases the score by 1
    self.score += 1

    # Updates the score display on the canvas
    self.canvas.itemconfig(self.score_text, text=f"Score = {self.score}")
    print(f"score {self.score}")

    # Checks the win condition for 10 correct answers and then calls the win_game function
    if self.score == 10:
        self.win_game()
        return

    # Plays the correct answer sound effect
    play_correct()
```

Result

Only one answer can now be processed per question.

Final Duck Jump Animation for Win

Problem

The duck did not always land properly at the end of the pond jump during the win sequence.

Cause

pond_jump() sometimes left the duck floating slightly above the target Y.

Buggy Code

```
# Applies gravity for a smooth jump arc
self.velocity_y += self.gravity
new_y = y + self.velocity_y

# Updates the duck's position on the screen
self.canvas.coords(self.sprite, new_x, new_y)
```

Fix

Set the Y coordinate to the target value when the animation completes.

Fixed Code

```
# Stops the duck's movement when it reaches the pond
if x >= target_x:
    self.canvas.coords(self.sprite, target_x, target_y)

    # Sets is moving flag to false
    self.is_moving = False

    print("POND JUMP FINISHED") # Debug code

    # Runs the end game function
    if on_finish:
        on_finish()

    return # Stops the animation loop
```

Result

The duck now lands exactly in the pond after the win jump.

Background Image Fails to Load

Problem

The background image does not appear if the file is missing or the path is incorrect.

Cause

`Image.open("background.png")` is called directly. If the file does not exist in the working directory, the program will throw an error and the background won't display.

Buggy Code

```
# Loads the background image and resizes to the window
self.bg_img = Image.open("bg.png").resize((700, 600))
self.bg_icon = ImageTk.PhotoImage(self.bg_img)

# Draws the background image to the canvas
background(self.canvas, self.bg_icon)
```

Fix

Ensure the image file "bg.png" exists in the correct directory and the correct file path is used.

```
# Creates the main canvas
self.canvas = tk.Canvas(root, width=500, height=500)
self.canvas.pack(fill="both", expand=True)
self.bg_icon = ImageTk.PhotoImage(Image.open("../img/bg.png"))

# Adds the background image
background(self.canvas, self.bg_icon)
```

Fixed Code

Ensure the file path is correct. *(added the ../img/ as the image was stored in the img file)*

Made sure the image was resized correctly to the correct window size.

Result

- The background displays correctly when the file exists and the correct file path is used.
- The background shows at the correct size for the window.
- The game layout renders correctly.
- If the file is missing, an error will display in the terminal.

Appendix 2

Debug Examples

1. Dry Run Example

Function: correct_answer()

Purpose: Ensure multiple clicks do not increase score incorrectly.

Steps:

1. Duck starts at self.duck = Duck(self.canvas, 32, 455, "../img/duck2.png")
2. Score = 0
3. Player clicks correct answer → correct_answer() runs:
 - Checks self.question_locked → False
 - Updated score = 1
 - Sets self.question_locked = True
 - Duck moves toward cracker → self.duck.move(45, self.cracker, on_collision=on_eaten)

Observation: Only one increment per question

The trace table below outlines the movement and positioning of the sprites (the duck and the cracker), also when the collision happens. It also includes the automation of the collision logic to show the cracker being hidden upon the triggering of on_eaten.

Trace Table – Duck Movement on Correct Answer

Function: correct_answer() → triggers self.duck.move(45, self.cracker, on_collision=on_eaten)

Step	Duck X	Duck Y	Cracker X	Cracker Y	Collision ?	Notes
1	32	455	91	447	No	Duck starts at specified x,y position, cracker is visible at first x,y position
2	37.8	455	91	447	No	Duck moves right, bouncing animation starts
3	43.6	455	91	447	No	Duck continues moving, velocity and gravity bouncing variables applied
4	45	455	91	447	Yes	Duck reaches cracker → collision detected, on_eaten() called
5	45	455	None	None	N/A	Cracker hidden after collision, feedback cleared, next question loaded

Using the duck movement code the columns show the position of the duck compared to the cracker and if a collision has taken place:

- Duck X / Duck Y: Current position of the duck on the canvas.
- Cracker X / Cracker Y: Current position of the cracker sprite. None after it is hidden.
- Collision?: Whether the duck collides with the cracker. Uses `check_collision()` logic